

A.3 Interfaz de usuario (UI) en entornos móviles

Si UX es la estructura biomecánica y el flujo de decisiones de una aplicación, la Interfaz de Usuario (UI) es la implementación técnica concreta de su capa visual y sensorial. En aplicaciones móviles, traducir maquetas de diseño a código no consiste únicamente en replicar píxeles; implica gestionar el rendimiento del motor gráfico, optimizar los ciclos de maquetado para evitar la pérdida de cuadros (*frame drops*), y construir sistemas de diseño escalables y mantenibles.

A.3.1 Sistemas de diseño y design tokens

- **Anatomía de un design system móvil:** Átomos, moléculas, organismos y plantillas (*Atomic Design* aplicado a componentes móviles).
- **Estructura de design tokens:** Abstracción de valores primitivos en variables reutilizables para Color (Semántico vs. Primitivo), Tipografía, Espaciado (*8pt Grid System*), Elevación y Radios de Borde.
- **Soporte nativo para temas dinámicos:** Mapeo de paletas para Modo Claro (*Light Mode*), Modo Oscuro (*Dark Mode*) y colores dinámicos del sistema operativo (Material You / Dynamic Colors).

A.3.2 Layouts, grids y maquetado adaptativo

- **Sistemas de posicionamiento móvil:** Flexbox, Box Constraints, Relative Layouts y Constraint Layouts.
- **Principios de adaptabilidad móvil:** Manejo de múltiples densidades de píxeles (dp/pt), proporciones de aspecto (*Aspect Ratios*) y áreas seguras del hardware (*Safe Areas / Notch / Punch Holes*).
- **Estrategias para pantallas plegables y comprimidas:** Layouts responsivos con puntos de quiebre (*Breakpoints*) para teléfonos, plegables (*Foldables*) y tabletas.

A.3.3 Motores de renderizado y optimización de rendimiento UI

- **Pipeline de renderizado:** El ciclo *Build -> Layout -> Paint -> Rasterize*.

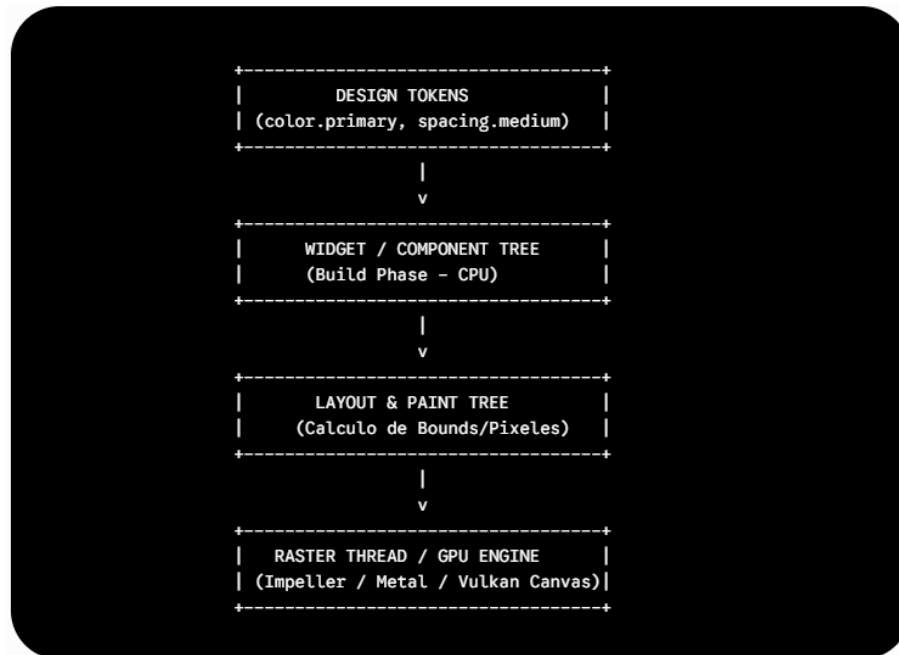
- **Arquitectura de motores gráficos:** Funcionamiento de Impeller/Skia (Flutter), Skiko, RenderThread (Android nativo) y Quartz/Metal (iOS).
- **Prevención de *jank* y pérdida de cuadros:** Métricas de renderizado (16.6 ms por cuadro para 60 FPS / 8.33 ms para 120 FPS), constante reinvocación de *draws* (*rebuids* innecesarios) y uso de widgets inmutables / const.

A.3.4 Micro-interacciones y animaciones técnicas

- **Animaciones implícitas vs. explícitas:** Transiciones automáticas de propiedades (*AnimatedContainer*) vs. Controladores de animación fina (*AnimationController*, *Ticker*).
- **Física de animaciones (*Spring Physics*):** Curvas de tiempo (*Easing*) frente a simulaciones basadas en masa, rigidez y amortiguación (*Stiffness & Damping*).
- **Animaciones basadas en vectores *lottie* y *rive*:** Integración de gráficos vectoriales interactivos ejecutados directamente en la GPU sin sobrecargar el hilo principal (*UI Thread*).

A.3.5 Fundamentos teóricos y términos clave

- **Design tokens:** La capa más atómica de un sistema de diseño. Son pares clave-valor agnósticos a la tecnología que almacenan decisiones visuales (ej. `color.bg.surface = #FFFFFF`). Permiten que plataformas web, iOS y Android compartan la misma identidad visual compilada desde una fuente única.
- **Densidad de píxeles independiente (dp/pt):** Unidad de medida abstracta que permite a la interfaz escalar de forma consistente en pantallas con diferentes concentraciones de píxeles por pulgada (DPI). En Android, 1 dp equivale a 1 px en una pantalla de 160 dpi.
- **Jank (Tirón de renderizado):** Incidencia que ocurre cuando el hilo de interfaz (*UI Thread*) tarda más del presupuesto asignado (16.6 ms) en procesar un cuadro. Esto provoca que la GPU repita el cuadro anterior, generando una experiencia visual pausada o "a saltos" durante el desplazamiento.
- **Shader warm-up / Shader compilation jank:** Fenómeno en el que la compilación de sombreadores en tiempo de ejecución genera tirones la primera vez que se renderiza una animación compleja. Motores modernos como Impeller resuelven esto compilando previamente (*AOT - Ahead Of Time*) los sombreadores.



A.3.6 Ejemplos prácticos y casos de estudio

Ejemplo A: Definición estructurada de tokens de diseño (JSON Schema)

Centralización de tokens para ser exportados automáticamente a código Flutter, Jetpack Compose o Swift.

```

{
  "color": {
    "brand": {
      "primary": { "value": "#0284C7", "type": "color" }
    },
    "semantic": {
      "surface": {
        "light": { "value": "{color.neutral.100}", "type": "color" },
        "dark": { "value": "{color.neutral.900}", "type": "color" }
      }
    }
  },
  "spacing": {
    "xs": { "value": "4dp", "type": "dimension" },
    "md": { "value": "16dp", "type": "dimension" }
  }
}

```

Ejemplo B: Redibujado innecesario vs. redibujado eficiente

Anti-patrón (Reconstrucción masiva del árbol de UI en cada pulso de reloj):

```
// Causa re-renderizado de toda la pantalla a 60 FPS
Widget build(BuildContext context) {
  return Scaffold(
    body: Column(
      children: [
        const HeavyStaticHeader(), // Se redibuja inútilmente
        AnimatedBuilder(
          animation: _controller,
          builder: (context, child) => Text('${_controller.value}'),
        ),
      ],
    ),
  );
}
```

Práctica Adecuada (Uso del parámetro child para aislar el sub-árbol estático):

```
Widget build(BuildContext context) {
  return Scaffold(
    body: Column(
      children: [
        const HeavyStaticHeader(), // Preservado en memoria mediante const
        AnimatedBuilder(
          animation: _controller,
          child: const StaticSubtree(), // No se vuelve a renderizar
          builder: (context, child) {
            return Row(
              children: [
                Text('${_controller.value}'),
                child!, // Pasa el sub-árbol precached
              ],
            );
          },
        ),
      ],
    ),
  );
}
```

A.3.7 Actividades práctica

Actividad 1: Profiling de renderizado y aislamiento de rebuilds

Identificar un cuello de botella de rendimiento gráfico y reestructurar el componente para eliminar la pérdida de cuadros.

1. Analicen un caso de código donde una lista de 1000 elementos renderiza íconos animados de "Me gusta", provocando que toda la lista se redibuje cuando un solo ícono cambia de estado.
2. Propongan la solución arquitectónica aislando el estado visual mediante componentes inmutables (const), delegación de estado local y uso del *Render Layer Tree*.
3. Calculen el presupuesto de tiempo por cuadro (16.6 ms) y expliquen cómo la solución evita sobrepasar dicho límite.

Solución:

Diagnóstico del Cuello de Botella (Jank en Listas)

- **Causa del antipatrón:** Cuando el usuario interactúa con el botón "Me gusta" de una tarjeta, la mutación de estado se eleva al ancestro común superior (p. ej., la lista completa). Esto invalida el contexto global y fuerza a ejecutar el método `build()` en los 1,000 elementos de la lista.
- **Impacto en el hilo de interfaz (UI Thread):** Procesar el maquetado de 1,000 widgets consume más de 45 ms en la CPU. Dado que el presupuesto para 60 FPS es de 16.66 ms por cuadro, la app descarta 2 o 3 cuadros consecutivos, generando tirones marcados (*jank*) durante el desplazamiento.

Solución Arquitectónica (Aislamiento de estado y sub-árbol inmutable)

Se aplica el principio de **encapsulamiento de estado local** y **optimización del árbol de renderizado**.

// ITEM DE LISTA OPTIMIZADO CON ESTADO LOCAL Y SUB-ÁRBOL INMUTABLE

```
class FavoriteItemTile extends StatelessWidget {
  final String itemId;
  final String title;
```

```
  // Constructor inmutable con clave de optimización
  const FavoriteItemTile({
    required this.itemId,
    required this.title,
    super.key,
  });
```

```
@override
Widget build(BuildContext context) {
  return ListTile(
    // Sub-árbol estático preservado mediante const
    title: Text(title),
    // Módulo aislado que gestiona de forma autónoma su re-renderizado
```

```

        trailing: FavoriteButtonWidget(itemId: itemId),
    );
}
}

// BOTÓN INDEPENDIENTE QUE MUTA SU ESTADO SIN REBUILD DEL PADRE
class FavoriteButtonWidget extends StatefulWidget {
  final String itemId;
  const FavoriteButtonWidget({required this.itemId, super.key});

  @override
  State<FavoriteButtonWidget> createState() => _FavoriteButtonWidgetState();
}

class _FavoriteButtonWidgetState extends State<FavoriteButtonWidget> {
  bool isLiked = false;

  void _toggleLike() {
    setState(() {
      isLiked = !isLiked; // Reconstruye ÚNICAMENTE este widget leaf (hoja)
    });
  }

  @override
  Widget build(BuildContext context) {
    return IconButton(
      icon: Icon(
        isLiked ? Icons.favorite : Icons.favorite_border,
        color: isLiked ? Colors.red : Colors.grey,
      ),
      onPressed: _toggleLike,
    );
  }
}

```

Análisis del Presupuesto de Renderizado (16.6 ms)

- **Consumo de CPU por cuadro (antes del aislamiento):**
 Tiempo Total = $T_{\text{Lista}(1000 \text{ items})} \approx 45.0 \text{ ms}$
 (Resultado: Jank, tasa final de $\sim 22 \text{ FPS}$)
- **Consumo de CPU por cuadro (después de la optimización):**
 Tiempo Total =
 $T_{\text{Rebuild}(1 \text{ IconButton})} + T_{\text{Rasterization}} \approx 1.2 \text{ ms} + 3.5 \text{ ms} = 4.7 \text{ ms}$
- **Conclusión térmica y gráfica:** Al limitar la invalidación del árbol a un nodo hoja (IconButton), el procesador opera holgadamente por debajo del límite de 16.66 ms ($4.7 \text{ ms} \ll 16.66 \text{ ms}$). Esto mantiene una tasa fluida y constante de 60 FPS (120 FPS en pantallas de alta frecuencia de refresco con presupuesto de 8.33 ms).